

Large-Scale Agentic Benchmark Evaluation: Comprehensive Execution Plan

1. Overview

Setup: 50 agentic benchmarks $\times$ ~ 100 tasks each $\times$ 2 agents $\times$ ~ 20 models (9 providers, ~ 2 models each) $\times$ 5 trials.

Dimension ordering (used throughout): Benchmark $\rightarrow$ Task $\rightarrow$ Agent $\rightarrow$ Model $\rightarrow$ Trial

Total tasks: $\sim 5,000$

Total runs: $\sim 5,000 \times 2 \times 20 \times 5 = \sim 1,000,000$

Data per task: ~ 200 data points (2 agents $\times$ 20 models $\times$ 5 trials)

Goals:

1. Cross-benchmark correlation and redundancy analysis
 2. Failure diagnosis and bottleneck identification via LLM trajectory evaluation
 3. Selection of HarborMix: a 100-task meta-benchmark
-

2. Related Work

2.1 Benchmark Compression and Efficient Evaluation

metabench (Kipnis et al., ICLR 2025) uses data from $n > 5,000$ LLMs across 6 benchmarks (ARC, GSM8K, HellaSwag, MMLU, TruthfulQA, WinoGrande) with 28,632 items total. Using point-biserial correlation and factor analysis, they distill a sparse benchmark at $< 3\%$ of original size, achieving Spearman $\rho = 0.94$ with full-suite rankings. Key finding: high correlations within and between benchmark scores suggest a small set of common underlying abilities.

- *What we learn:* Point-biserial filtering + factor analysis is well-validated for item selection.
- *How we go beyond:* metabench covers 6 static knowledge/reasoning benchmarks. We cover 50 agentic benchmarks with trajectory data, agent $\times$ model interactions, and repeated trials.
- *Citation:* <https://openreview.net/forum?id=4T33izzFpK>

tinyBenchmarks (Polo et al., ICML 2024) uses IRT to select 100 representative items per benchmark, achieving $< 2\%$ estimation error on MMLU, Open LLM Leaderboard, HELM, and AlpacaEval. Their IRT++ variant (IRT + Gaussian process regression) consistently outperforms vanilla IRT.

- *What we learn:* IRT + Fisher information is a principled item selection criterion.

- *How we go beyond:* tinyBenchmarks compresses *within* a single benchmark. We compress *across* 50 benchmarks — a fundamentally harder problem.
- *Citation:* <https://arxiv.org/abs/2402.14992>

PSN-IRT / "Lost in Benchmarks" (Zhou et al., 2025) proposes enhanced neural IRT applied to 11 benchmarks and 41,871 items. Key cautionary finding: selecting items by discriminability alone fails to differentiate strong models — it only separates capability tiers. Clustering items by success/failure vectors also fails to improve ranking correlation.

- *What we learn:* Pure discrimination-based selection is insufficient. Must combine discrimination with difficulty diversity and latent space coverage.
- *Citation:* <https://arxiv.org/abs/2505.15055>

ATLAS (Zhuang et al., 2026) introduces adaptive testing for LLMs — selecting items dynamically based on responses. Outperforms both tinyBenchmarks and metabench on psychometric fit. Key insight: partition-based calibration enables reliable validation.

- *What we learn:* Report IRT model-fit diagnostics (M_2 statistic, RMSEA) — prior work often skips this.
- *Citation:* <https://arxiv.org/abs/2511.04689>

SparseEval (2025) frames evaluation efficiency as exploiting sparsity in the (model $\times$ item) response matrix, complementary to BenchPress's low-rank observation.

- *Citation:* <https://arxiv.org/abs/2602.07909>

2.2 Benchmark Agreement and Correlation

BenchBench (Perlitz et al., 2024) analyzes 40+ benchmarks and shows that Benchmark Agreement Testing (BAT) conclusions are highly sensitive to: which models are included, which reference benchmark is used, and the correlation metric. Including too many weak models inflates correlations artificially.

- *What we learn:* Report correlations with all 20 models AND top-10 frontier models separately. Use Spearman, report confidence intervals.
- *Citation:* <https://arxiv.org/abs/2407.13696>

BenchPress (Papailiopoulos, 2026) demonstrates the (model $\times$ benchmark) score matrix is approximately rank-2, and 5 benchmarks predict the other 44 within 7% median error. Two latent components: "general capability" and "frontier reasoning recency." Logit transform improves prediction ~10%.

- *What we learn:* Low-rank structure. Metadata (param count, provider) doesn't help beyond what scores capture.
- *How we go beyond:* We have task-level granularity, trajectory data, and a 3D tensor (not 2D matrix).
- *Citation:* <https://github.com/anadim/llm-benchmark-matrix>

2.3 Agent Failure Analysis

Terminal-Bench 2.0 (Merrill et al., ICLR 2026) provides the gold standard for trajectory error analysis. Terminal Agent Taxonomy (TAT): 3 classes (Execution ~55%, Coherence ~25%, Verification ~20%) with 8 subcategories. Also: command-level analysis with ~100-leaf taxonomy. GPT-5 high-reasoning achieves 90% agreement with human labels.

- *What we learn:* Two-level analysis (trajectory + step) is essential. Per-mode rubrics outperform multi-label prompts.
- *How we go beyond:* TB 2.0 is terminal-specific, single-agent. We extend to 15 subcategories across 50 diverse benchmarks with 2 agents.
- *Citation:* <https://arxiv.org/abs/2601.11868>

MAST (Pan et al., 2025) defines a comprehensive multi-agent system failure taxonomy. TB 2.0 simplifies MAST for single-agent settings. Many MAST categories (conversation reset, information withholding) don't apply.

- *Citation:* Pan et al. (2025). "Why do multiagent systems fail?"

Evaluation and Benchmarking of LLM Agents: A Survey (Mohammadi et al., KDD 2025) provides a two-dimensional taxonomy of agent evaluation: what to evaluate × how to evaluate. Identifies gaps: "holistic, more realistic, and scalable evaluation."

- *Our work fills this exact gap.*
- *Citation:* <https://arxiv.org/abs/2507.21504>

2.4 What's Novel About Our Work

Dimension	Best prior work	Our study
# Benchmarks	6 (metabench), 11 (PSN-IRT)	50 agentic benchmarks
# Tasks	28,632 (metabench)	~5,000 (agentic, not MCQ)
Agent dimension	None (all prior work ignores scaffold)	2 agents → interaction analysis

Dimension	Best prior work	Our study
Repeated trials	None (single run)	5 trials → reliability analysis
Trajectory data	TB 2.0 (1 benchmark, 1 agent)	50 benchmarks, 2 agents
Analysis depth	Score-only (IRT, correlation)	Score + trajectory + step-level + failure taxonomy
Meta-benchmark	Within-benchmark (tinyBenchmarks)	Cross-benchmark 100-task selection
Decomposition	2D matrix (BenchPress)	3D tensor (benchmark × agent × model)

3. Data Schema

Dimension ordering: **Benchmark → Task → Agent → Model → Trial**

```

runs/
  run_id: str
  benchmark: str
  task_id: str          # globally unique: "{benchmark}__{task_name}"
  agent: str            # agent_A | agent_B
  provider: str         # openai, anthropic, google, ...
  model: str            # gpt-5.2, claude-opus-4.6, ...
  trial: int            # 1..5
  success: bool
  score: float          # [0,1] if partial credit, else 0/1
  trajectory: str       # full trajectory text or path
  num_steps: int
  num_tokens_input: int
  num_tokens_output: int
  num_tool_calls: int
  wall_time_s: float
  error_type: str | null # timeout | crash | wrong_answer | null

tasks/
  task_id: str
  benchmark: str
  category: str | null
  description: str
  reference_answer: str | null
  estimated_difficulty: str | null

models/
  model: str
  provider: str
  param_count_B: float | null
  is_reasoning: bool
  is_open_weight: bool
  release_date: date | null

```

All code uses `.groupby(['benchmark', 'task_id', 'agent', 'model', 'trial'])` ordering.

4. Phase 1: Descriptive Statistics and Data Quality

Purpose

Validate data completeness, characterize the dataset, establish baselines.

Execution

1.1 Completeness tensor. Build a 4D tensor (benchmark/task × agent × model × trial) and check fill rate. Report per-benchmark completeness.

1.2 Per-task summary statistics. For each task ($n \approx 5,000$):

```
python

task_stats = {
    'pass_rate': mean success across all 200 data points,
    'trial_std_mean': mean(std across 5 trials for each (agent, model)),
    'model_range': max - min pass rate across models (averaged over agents),
    'agent_gap': |pass_rate(agent_A) - pass_rate(agent_B)| averaged across models,
    'mean_steps': mean num_steps across all runs,
    'mean_tokens': mean total tokens across all runs,
}
```

1.3 Difficulty distribution. Bin tasks by pass rate:

- **Frontier** (< 0.05): unsolved — *highly interesting*
- **Hard** (0.05-0.30): only best systems solve
- **Medium** (0.30-0.70): discriminative range
- **Easy** (0.70-0.95): most systems solve
- **Saturated** (> 0.95): nearly all solve

Important: Do NOT filter any tier. All tasks are kept:

- Frontier/hard → capability frontier analysis, prioritized for HarborMix
- Easy/saturated → useful for benchmark correlation analysis
- For HarborMix, deprioritize (not remove) saturated tasks

1.4 Trial consistency. Per (benchmark, task, agent, model): std across 5 trials.

Expected Results

- ~5-10% of tasks are frontier, ~15-20% hard, ~30-40% medium, ~20-25% easy, ~10-15% saturated
- Mean trial std ≈ 0.15 -0.25 (moderate stochasticity in agentic tasks)
- Agent gap will vary by benchmark: near zero for pure reasoning, larger for tool-heavy tasks

Output

- Data quality report, tier distributions per benchmark, full task_stats table

5. Phase 2: Variance Decomposition

Purpose

Answer: what matters more — the benchmark, the agent, or the model?

Method: Multi-factor ANOVA

What it is: Decomposes total variance into contributions from factors and interactions.

Reference: Gelman & Hill (2006). *Data Analysis Using Regression and Multilevel/Hierarchical Models*.

$$\begin{aligned} \text{score} = & \mu + \alpha(\text{benchmark}) + \beta(\text{agent}) + \gamma(\text{model}) \\ & + (\alpha\beta) + (\alpha\gamma) + (\beta\gamma) + (\alpha\beta\gamma) + \varepsilon(\text{trial}) \end{aligned}$$

Execution

```
python

import statsmodels.api as sm
from statsmodels.formula.api import ols

df_agg = runs_df.groupby(['benchmark', 'agent', 'model']).agg(
    pass_rate=('success', 'mean')
).reset_index()

model_formula = 'pass_rate ~ C(benchmark) * C(agent) * C(model)'
lm = ols(model_formula, data=df_agg).fit()
anova_table = sm.stats.anova_lm(lm, typ=2)
anova_table['eta_sq'] = anova_table['sum_sq'] / anova_table['sum_sq'].sum()
```

Expected Results and Interpretation

Factor	If $\eta^2 \approx 0.60-0.70$	If $\eta^2 \approx 0.30-0.40$	If $\eta^2 \approx 0.05-0.15$
model	Expected. "Pick the right model; the agent barely matters."	Surprisingly low. "Models have converged; agent design is the differentiator."	Very surprising.

Factor	If $\eta^2 \approx 0.60\text{--}0.70$	If $\eta^2 \approx 0.30\text{--}0.40$	If $\eta^2 \approx 0.05\text{--}0.15$
agent	Headline finding: agent architecture matters enormously.	Moderate. "Agent design has meaningful but not dominant impact."	Expected. Agent is secondary.
model×agent	Most interesting. "No single best pairing; must co-optimize."	Small interaction. Effects are mostly separable.	"Just pick the best model and best agent independently."
model×benchmark	"Models are specialists."	Some specialization.	"A good model is good at everything" — consistent with rank-2.

Predicted story: Based on TB 2.0 ("model selection is usually more important than agent scaffold"), we expect model $\eta^2 \approx 0.50\text{--}0.65$, agent $\eta^2 \approx 0.05\text{--}0.10$, model×agent $\approx 0.05\text{--}0.10$. If the agent effect is larger, that's a strong signal for the community to invest more in agent architecture.

Output

- ANOVA table with η^2 for each factor
- Variance attribution bar chart

6. Phase 3: Tensor Decomposition

Purpose

Find latent structure of (benchmark × agent × model) scores. How many independent capability dimensions exist?

Method: Tucker Decomposition

What it is: Generalizes SVD to 3+ dimensions. $T \approx G \times_1 A \times_2 B \times_3 C$. Rank along each mode = intrinsic dimensionality.

Reference: Kolda & Bader (2009). *SIAM Review*, 51(3). <https://doi.org/10.1137/07070111X>

Library: `tensorly` — <https://tensorly.org/>

Execution

```
python
```



```

import tensorly as tl
from tensorly.decomposition import tucker
import numpy as np

# Tensor shape: (n_benchmarks=50, n_agents=2, n_models=20)
tensor = np.zeros((n_benchmarks, n_agents, n_models))
# Fill: cell = mean pass rate across tasks, averaged over 5 trials

results = []
for r_bench in range(1, 8):
    for r_agent in [1, 2]:
        for r_model in range(1, 8):
            core, factors = tucker(tensor, rank=[r_bench, r_agent, r_model])
            recon = tl.tucker_to_tensor((core, factors))
            rmse = np.sqrt(np.mean((tensor - recon) ** 2))
            results.append({'r_bench': r_bench, 'r_agent': r_agent,
                           'r_model': r_model, 'rmse': rmse})

```

Expected Results and Interpretation

Mode	If rank ≈ 1	If rank ≈ 2	If rank $\approx 3-5$
Benchmark	All benchmarks measure one thing. Extreme redundancy.	Expected (matches BenchPress). "50 benchmarks $\rightarrow$ 2 dimensions."	More capability dimensions than BenchPress — perhaps agentic benchmarks are more diverse than standard evals.
Model	All models identical up to scalar.	Expected. Two dimensions: "capability" and "reasoning vs breadth."	Complex multi-dimensional profiles.
Agent	Rank 1 = null hypothesis. Agent is a scalar multiplier.	Rank 2 = agents have different profiles. "Agent A excels at planning-heavy tasks; Agent B at tool-intensive tasks."	N/A (max 2).

What factor loadings reveal:

- Plot benchmarks in 2D factor space, color by category $\rightarrow$ do coding/web/tool benchmarks cluster?
- Plot models in 2D, color by provider $\rightarrow$ do providers cluster?

- Factor 1 likely = "general agentic capability"; Factor 2 likely = "execution vs reasoning"

Output

- Effective rank per mode with reconstruction error curves
 - Factor loading visualizations (benchmarks, models in latent space)
-

7. Phase 4: Benchmark Correlation and Redundancy

Purpose

Which benchmarks measure the same thing? Which are unique?

Method 4.1: Spearman Correlation Matrix

Caution from BenchBench: Including many weak models inflates correlations. Report results with all models AND top-10 frontier only.

```
python

from scipy.stats import spearmanr
import seaborn as sns

# Rows = (agent, model) pairs (~40), columns = benchmarks (50)
corr_matrix, _ = spearmanr(score_matrix, axis=0)
sns.clustermap(corr_matrix, method='ward', cmap='RdBu_r', vmin=-1, vmax=1)
```

Method 4.2: Predictability Score (R^2)

Per benchmark B, fit ridge regression from all other benchmarks $\rightarrow$ B. Low R^2 = unique.

```
python
```

```
from sklearn.linear_model import RidgeCV
from sklearn.model_selection import cross_val_score

predictability = {}
for target in benchmarks:
    X = score_matrix.drop(columns=[target]).values
    y = score_matrix[target].values
    mask = ~np.isnan(y)
    model_ridge = RidgeCV(alphas=[0.1, 1, 10, 100])
    scores = cross_val_score(model_ridge, X[mask], y[mask], cv=5, scoring='r2')
    predictability[target] = {'r2_mean': scores.mean(), 'r2_std': scores.std()}
```

Method 4.3: Agent-Conditional Correlation

Run 4.1 and 4.2 separately per agent. If two benchmarks correlate $r=0.9$ under agent_A but $r=0.5$ under agent_B, the agent architecture reveals a distinction.

Method 4.4: Task-Level Cross-Benchmark Correlation

Compute Φ coefficient between pass/fail vectors for task pairs from *different* benchmarks. Build similarity graph. Connected components = functionally equivalent tasks across benchmarks.

Expected Results

R ² range	Meaning	Expected count	Examples
> 0.90	Highly redundant	~10-15	Easy coding benchmarks correlating with each other
0.70-0.90	Moderately predictable	~20-25	Medium-difficulty general benchmarks
0.40-0.70	Partially unique	~8-12	Specialized (security, scientific computing)
< 0.40	Genuinely unique	~3-5	Frontier/novel capability tests

Story: "Of 50 agentic benchmarks, only ~5 are genuinely unique. HarborMix prioritizes these."

Output

- Clustered heatmap, predictability ranking, agent-differential table, task similarity graph

8. Phase 5: Item Response Theory (IRT)

Purpose

Assign each task difficulty and discrimination scores for principled HarborMix selection.

Method: 2-Parameter Logistic (2PL) IRT

What it is: Models $P(\text{pass})$ as function of respondent ability and item properties. From psychometrics.

Reference: Baker & Kim (2017). *The Basics of IRT Using R*. Springer.

Important (from ATLAS): Report model-fit diagnostics (M_2 , RMSEA) — prior work often omits this.

$$P(\text{pass}) = 1 / (1 + \exp(-a_j \times (\theta_i - b_j)))$$

```
python

# py-irt: pip install py-irt
from py_irt.training import IRT_Training

responses = []
for system_id in systems: # system = (agent, model) pair
    for task_id in tasks:
        n_pass = sum(runs[(task_id, system_id)].success)
        correct = 1 if n_pass >= 3 else 0 # majority vote of 5 trials
        responses.append({'subject_id': system_id, 'item_id': task_id, 'correct': correct})

training = IRT_Training(data=responses, model_type='2pl', epochs=2000)
training.train()
params = training.get_params()
```

Differential Item Functioning (DIF)

What it is: Tests whether a task functions differently for agent_A vs agent_B after controlling for ability.

Reference: Holland & Wainer (1993). *Differential Item Functioning*.

```
python

# Fit separate IRT for agent_A and agent_B
# DIF = |b_j(agent_A) - b_j(agent_B)|
# Tasks with large DIF are agent-sensitive
```

Expected Results

Parameter	Expected range	Interpretation
Difficulty (b)	-3 to +3	$b > 2$: only strongest systems pass. $b < -2$: trivially easy.
Discrimination (a)	0.3 to 3.0	$a > 1.5$: excellent discriminator (select for HarborMix). $a < 0.5$: noisy. $a < 0$: aberrant.
Point-biserial	-0.2 to 0.8	> 0.3 : good. < 0.1 : noisy. < 0 : investigate.

Key findings to look for:

- Tasks with $a > 2$: "razor-sharp discriminators" — gold for HarborMix
- Tasks with $a < 0$: "aberrant" — weak models beat strong ones. Investigate why (ambiguous criteria? evaluation noise?)
- Tasks with high DIF: agent architecture specifically matters. Expected: 5-15% of tasks. Likely in planning-heavy or tool-heavy benchmarks.

Caution from PSN-IRT: Discrimination-based selection alone fails for top-model differentiation. Our approach combines discrimination + difficulty diversity + cluster coverage + frontier bonus.

Output

- Per-task table: (task_id, benchmark, difficulty, discrimination, point_biserial, trial_std, DIF)
- Scatter: difficulty vs discrimination, colored by benchmark
- Lists of aberrant and high-DIF tasks

9. Phase 6: LLM Trajectory Evaluation

Purpose

Classify *why* agents fail and *where* in trajectories failures occur.

References

- Terminal-Bench 2.0 (Merrill et al., 2026): Appendix C (TAT) + Appendix E (command taxonomy)
- MAST (Pan et al., 2025): multi-agent failure taxonomy

- Docent (Meng et al., 2025): LLM annotation pipeline

6.1 Extended Failure Taxonomy: 3 classes, 15 subcategories

TB 2.0 has 3 classes / 8 subcategories for terminal tasks. We extend to 15 subcategories for 50 diverse agentic benchmarks:

EXECUTION (the agent does the wrong thing)

- └─ E1. Disobey Task Specification [TB 2.0]
- └─ E2. Step Repetition / Loop [TB 2.0]
- └─ E3. Unaware of Termination Conditions [TB 2.0]
- └─ E4. Wrong Tool or Action Selection [NEW: for diverse benchmarks]
- └─ E5. Code / Command Generation Error [from TB 2.0 Appendix E]
- └─ E6. Environment Interaction Failure [NEW: web, API, filesystem]

COHERENCE (the agent loses track)

- └─ C1. Context Loss [TB 2.0]
- └─ C2. Reasoning-Action Mismatch [TB 2.0]
- └─ C3. Task Derailment [TB 2.0, extended]
- └─ C4. Hallucination / Fabrication [NEW: critical for web/knowledge]
- └─ C5. Premature Termination [TB 2.0]

VERIFICATION (the agent doesn't check its work)

- └─ V1. No or Incorrect Verification [TB 2.0]
- └─ V2. Weak Verification [TB 2.0]
- └─ V3. False Positive Self-Assessment [NEW]
- └─ V4. Ignored Error Recovery [NEW]

6.2 Trajectory-Level Evaluator Prompt

SYSTEM:

You are an expert evaluator analyzing an AI agent's trajectory on a task.

USER:

Task Description

{task_description}

Expected Outcome

{reference_answer_or_criteria}

Agent Trajectory

{trajectory_text}

Outcome

Result: {"SUCCESS" | "FAILURE"}

Instructions

Analyze this trajectory and respond with JSON:

```
{
  "outcome_correct": true/false,
  "primary_class": "EXECUTION" | "COHERENCE" | "VERIFICATION" | "NONE",
  "primary_failure_mode": one of [E1-E6, C1-C5, V1-V4, "NONE"],
  "secondary_failure_mode": one of the above or "NONE",
  "failure_step": <int, 1-indexed> | null,
  "trajectory_quality_score": <int, 0-100>,
  "trajectory_length_steps": <int>,
  "explanation": "<2-3 sentences>"
}
```

[Full definitions of E1-E6, C1-C5, V1-V4 included in prompt – see v2 plan for complete text]

6.3 Per-Mode Confirmation Rubrics

For top-3 failure modes, create targeted rubrics (following TB 2.0 Appendix C.2) with inclusion criteria, exclusion criteria, and evidence extraction. Example for E2 (Step Repetition):

Does this trajectory exhibit Step Repetition?

INCLUSION: Same sub-goal, tool, target, method repeated 3+ times without strategy change.

EXCLUSION: Retrying with different approach is NOT repetition.

Respond: {"exhibits_E2": true/false, "evidence": "<quote repeated steps>"}

6.4 Step-Level Error Analysis

Analogous to TB 2.0 Appendix E command-level analysis, generalized to any agent action:

```

## Step {step_number}
Agent action: {action}
Environment response: {response}
Exit code: {exit_code}

Classify error: COMMAND_NOT_FOUND | EXECUTION_ERROR | SYNTAX_ERROR |
PERMISSION_ERROR | PATH_FILE_ERROR | DEPENDENCY_ERROR | NETWORK_ERROR |
API_ERROR | OUTPUT_MISPARE | NO_ERROR

Respond: {"has_error": true/false, "error_type": "...", "agent_caused":
true/false}

```

6.5 Sampling Strategy

Category	Estimate	Action	Priority
Failed runs	~600k	Sample 50k (stratified by benchmark × agent × model)	HIGH
Successful runs	~400k	Sample 10k (false positive detection)	MEDIUM
Frontier tasks (pass_rate < 0.05)	All runs	Evaluate all failures	HIGH
High-variance tasks	All runs	Evaluate all	MEDIUM

Cost: ~\$200 total using cheap model. 5% validation with strong model. Target: Cohen's $\kappa > 0.80$.

Expected Results

Failure mode	Expected %	If higher →	If lower →
E1 (Disobey Spec)	10-15%	Models don't follow instructions	Good instruction following, fail elsewhere
E2 (Loop)	8-12%	Agent scaffolding needs loop detection	Good loop handling
E5 (Code Error)	15-20%	Code quality is the bottleneck	Planning, not coding, is the problem
C1 (Context Loss)	8-12%	Long-context problem	Good context handling

Failure mode	Expected %	If higher →	If lower →
C4 (Hallucination)	5-10%	Fabrication problem, especially in web/knowledge tasks	—
C5 (Premature Term.)	8-12%	Agent gives up too early	Agent is persistent

Cross-benchmark stories:

- Coding benchmarks with 30%+ E5 → "Bottleneck is code quality, not planning"
- Web benchmarks with 20%+ C1 → "Web tasks cause context loss"
- If smaller models have 2× more C4 → "Scale reduces hallucination in agentic settings"

Agent-differential stories:

- If Agent A has more E2 (loops), Agent B has more C5 (premature termination) → "Agent A is persistent but gets stuck; Agent B gives up but doesn't loop"

6.6 Aggregation

Per benchmark: failure mode distribution → reveals what benchmark *actually* tests

Per model: failure profile → cluster models by failure pattern

Agent differential: Jensen-Shannon divergence between agents' failure distributions

Bottleneck CDF: normalized failure_step position → planning vs execution vs verification bottleneck

Trajectory quality on frontier tasks: high quality + low pass rate = "almost solvable, needs marginal improvement"

Output

- Per-model and per-benchmark failure mode charts (stacked bar, like TB 2.0 Fig. 8)
- Bottleneck CDFs, agent differential table, frontier task characterization

10. Phase 7: Efficiency and Pass@k Analysis

Purpose

Measure reliability and efficiency beyond binary pass/fail.

Method: Pass@k

Reference: Chen et al. (2021). Evaluating Large Language Models Trained on Code.
<https://arxiv.org/abs/2107.03374>

```
python

from math import comb

def pass_at_k(n, c, k):
    if n - c < k: return 1.0
    return 1.0 - comb(n - c, k) / comb(n, k)

# Per (benchmark, task, agent, model): n=5, c=successes
```

Expected Results

Profile	pass@1	pass@5	Expected %	Story
Solid	>0.6	>0.8	20-30%	Reliably within capability
Fragile	<0.3	>0.6	15-25%	Can solve but usually doesn't. Agent improvements matter most here.
Gap	<0.2	<0.3	30-40%	Lacks capability. More trials won't help.
Saturated	>0.9	1.0	10-20%	Too easy.

Key metric: fragile-to-solid ratio. If > 0.5, agentic performance is highly stochastic. Expected: 0.3-0.5.

Efficiency Metrics

```
python

# Among successful runs: median steps, tokens, tool calls, wall time
# Pareto frontier: accuracy vs 1/median_tokens
```

Output

- Pass@k curves per model, profile distributions per benchmark, Pareto frontier

Goal

Select 100 tasks from ~5,000 maximizing representativeness.

Step 8.1: Task Score Matrix

```
python

# T: rows = tasks (~5,000), columns = (agent, model) systems (~40)
# Cell = pass@1 from 5 trials
T = runs_df.groupby(['task_id', 'agent', 'model'])['success'].apply(
    lambda x: pass_at_k(5, x.sum(), 1)
).unstack(['agent', 'model'])
```

Step 8.2: Tag tasks by tier (no filtering)

Step 8.3: Attach IRT parameters from Phase 5

Step 8.4: Task embeddings via SVD (d=5)

Step 8.5: Cluster tasks (K=60)

Step 8.6: Constrained Greedy Selection

```

def select_harbormix(tasks_df, n_select=100):
    selected = []
    remaining = set(tasks_df.index)

    # Phase 1: 1 task per benchmark (50 tasks) – highest discrimination, prefer non-
    for bench in tasks_df['benchmark'].unique():
        candidates = tasks_df[(tasks_df['benchmark'] == bench) & (tasks_df['tier'] != 'frontier')]
        if len(candidates) == 0:
            candidates = tasks_df[tasks_df['benchmark'] == bench]
        best = candidates.sort_values('discrimination', ascending=False).iloc[0]
        selected.append(best.name)
        remaining.discard(best.name)

    # Phase 2: Cover uncovered latent clusters
    covered_clusters = set(tasks_df.loc[selected, 'cluster'])
    for c in range(K):
        if c not in covered_clusters and len(selected) < n_select:
            cluster_tasks = tasks_df[(tasks_df['cluster'] == c) & (tasks_df.index.isin(selected))]
            if len(cluster_tasks) > 0:
                best = cluster_tasks.sort_values('discrimination', ascending=False).iloc[0]
                selected.append(best.name)
                remaining.discard(best.name)

    # Phase 3: Greedy fill – maximize discrimination × reliability × diversity × frontier bonus
    while len(selected) < n_select and remaining:
        current_difficulties = tasks_df.loc[selected, 'difficulty'].values
        current_tiers = tasks_df.loc[selected, 'tier'].value_counts()
        best_score, best_task = -1, None

        for t in remaining:
            disc = tasks_df.loc[t, 'discrimination']
            reliability = 1 - tasks_df.loc[t, 'trial_std_mean']
            min_dist = np.min(np.abs(current_difficulties - tasks_df.loc[t, 'difficulty']))
            tier_bonus = 1.0 / (1 + current_tiers.get(tasks_df.loc[t, 'tier'], 0))
            frontier_bonus = 2.0 if tasks_df.loc[t, 'tier'] in ['frontier', 'hard'] else 1.0
            score = disc * reliability * (1 + min_dist) * tier_bonus * frontier_bonus
            if score > best_score:
                best_score, best_task = score, t

        selected.append(best_task)
        remaining.discard(best_task)

    return selected

```

Step 8.7: Validation

```
python

from scipy.stats import kendalltau, spearmanr

full_ranking = T.mean(axis=0)
mix_ranking = T.loc[selected_tasks].mean(axis=0)
tau, _ = kendalltau(full_ranking, mix_ranking)
rho, _ = spearmanr(full_ranking, mix_ranking)
```

Baselines:

1. Random 100 (50 repeats, report mean $\pm$ std τ)
2. Stratified random: 2 per benchmark
3. Top-100 by discrimination
4. Greedy forward selection on τ

Expected Results

Metric	Target	If lower
Kendall τ	> 0.90	Need more tasks from low- R^2 benchmarks
Spearman ρ	> 0.95	Significant ranking disagreements
$\Delta\tau$ vs random	+0.10-0.15	Method barely helps $\rightarrow$ revisit
$\Delta\tau$ vs stratified	+0.05-0.10	Principled selection marginally better

Headline: "100 tasks preserve system rankings with $\tau > 0.90$. Running $\sim 5,000$ tasks provides marginal additional ranking information. However, [specific benchmarks] resist prediction and should be run independently."

Output

- HarborMix 100-task set with metadata
- Validation metrics vs baselines
- Per-benchmark prediction R^2

12. Phase 9: Cross-Benchmark Capability Mapping

Method 9.1: Factor Analysis

Reference: Fabrigar & Wegener (2012). *Exploratory Factor Analysis*.

```
python

from sklearn.decomposition import FactorAnalysis

fa = FactorAnalysis(n_components=5, random_state=42)
fa.fit(B_standardized) # B: rows=systems, columns=benchmarks
loadings = pd.DataFrame(fa.components_.T, index=benchmark_names,
                        columns=[f'Factor_{i+1}' for i in range(5)])
```

Method 9.2: Model Strength/Weakness Profiles

Z-score each model across benchmarks, cluster by profile.

Method 9.3: Agent Differential

Per benchmark: $\Delta = \text{score}(\text{agent_A}) - \text{score}(\text{agent_B})$ averaged across models. Benchmarks with large $|\Delta|$ = agent choice matters.

Output

- Factor loading table (name factors by top-loading benchmarks)
 - Model cluster profiles, agent differential ranking
-

13. Phase 10: Scaling and Provider Analysis

Method 10.1: Scaling Curves

Reference: Kaplan et al. (2020). *Scaling Laws for Neural Language Models*.

<https://arxiv.org/abs/2001.08361>

```
python

def scaling_law(params_B, a, b, c):
    return a - b * np.power(params_B, -c)
```

Expected shapes:

- Smooth power law ($R^2 > 0.9$): capability scales predictably

- Phase transition (sharp jump at ~30B): capability unlocking threshold
- Poor fit ($R^2 < 0.5$): training recipe matters more than scale

Method 10.2: Provider Fingerprinting

```
python

from sklearn.metrics import silhouette_score
sil = silhouette_score(benchmark_profiles, provider_labels)
# > 0.5: providers have distinctive profiles
# < 0.2: model size matters more than provider
```

Output

- Scaling curves per benchmark, phase transition analysis, provider clustering

14. Execution Timeline

Week	Phase	Deliverable
1	Phase 1: Data quality	Completeness report, task stats, tier distributions
1	Phase 2: ANOVA	Variance attribution chart
2	Phase 3: Tucker decomposition	Effective rank per mode, factor plots
2	Phase 4: Correlation	Clustered heatmap, predictability ranking
3	Phase 5: IRT	Per-task difficulty/discrimination, DIF
3-4	Phase 6: Trajectory evaluation	Failure taxonomy, bottleneck CDFs
4	Phase 7: Efficiency	Pass@k curves, Pareto frontiers
5	Phase 8: HarborMix	100-task selection, validation
5	Phase 9: Capability map	Factor loadings, profiles
6	Phase 10: Scaling	Scaling curves, provider fingerprints
6	Final	Integrated report, HarborMix release

15. Summary of Outputs

#	Output	Phase
1	Variance attribution: benchmark vs agent vs model vs interactions	2
2	Latent dimensionality: effective rank per tensor mode	3
3	Benchmark correlation map: redundant vs unique benchmarks	4
4	IRT parameters: per-task difficulty and discrimination	5
5	Failure taxonomy: 15-category classification, per (benchmark, agent, model)	6
6	Bottleneck analysis: where in trajectories failures occur	6
7	Step-level error distribution across models	6
8	Pass@k profiles: reliability characterization	7
9	HarborMix: 100-task meta-benchmark with validation	8
10	Capability map: factor loadings, model profiles	9
11	Scaling analysis: power-law fits, phase transitions, provider clustering	10

16. References

Method	Reference	Link
ANOVA	Gelman & Hill (2006). <i>Data Analysis Using Regression</i>	—
Tucker decomposition	Kolda & Bader (2009). <i>SIAM Review</i> 51(3)	https://doi.org/10.1137/07070111X
Matrix completion / SVD	Candès & Recht (2009). <i>Found. Comput. Math.</i>	https://doi.org/10.1007/s10208-009-9045-5
BenchPress	Papailiopoulos (2026)	https://github.com/anadim/llm-benchmark-matrix

Method	Reference	Link
metabench	Kipnis et al. (ICLR 2025)	https://openreview.net/forum?id=4T33izzFpK
tinyBenchmarks	Polo et al. (ICML 2024)	https://arxiv.org/abs/2402.14992
PSN-IRT	Zhou et al. (2025)	https://arxiv.org/abs/2505.15055
ATLAS	Zhuang et al. (2026)	https://arxiv.org/abs/2511.04689
BenchBench	Perlitz et al. (2024)	https://arxiv.org/abs/2407.13696
SparseEval	(2025)	https://arxiv.org/abs/2602.07909
IRT (2PL)	Baker & Kim (2017). <i>Basics of IRT</i>	—
DIF	Holland & Wainer (1993)	—
Pass@k	Chen et al. (2021). <i>Evaluating LLMs on Code</i>	https://arxiv.org/abs/2107.03374
Terminal Agent Taxonomy	Merrill et al. (ICLR 2026). <i>Terminal-Bench 2.0</i>	https://arxiv.org/abs/2601.11868
MAST	Pan et al. (2025). <i>Why do multiagent systems fail?</i>	—
Agent eval survey	Mohammadi et al. (KDD 2025)	https://arxiv.org/abs/2507.21504
Factor analysis	Fabrigar & Wegener (2012). <i>Exploratory FA</i>	—
Scaling laws	Kaplan et al. (2020)	https://arxiv.org/abs/2001.08361
tensorly	Kossaifi et al. (2019). <i>JMLR</i> 20(26)	https://tensorly.org/
py-irt	—	https://github.com/nd-ball/py-irt
girth	—	https://github.com/eribean/girth